BUSINESS INTELLIGENCE AND DATA ANALYTICS

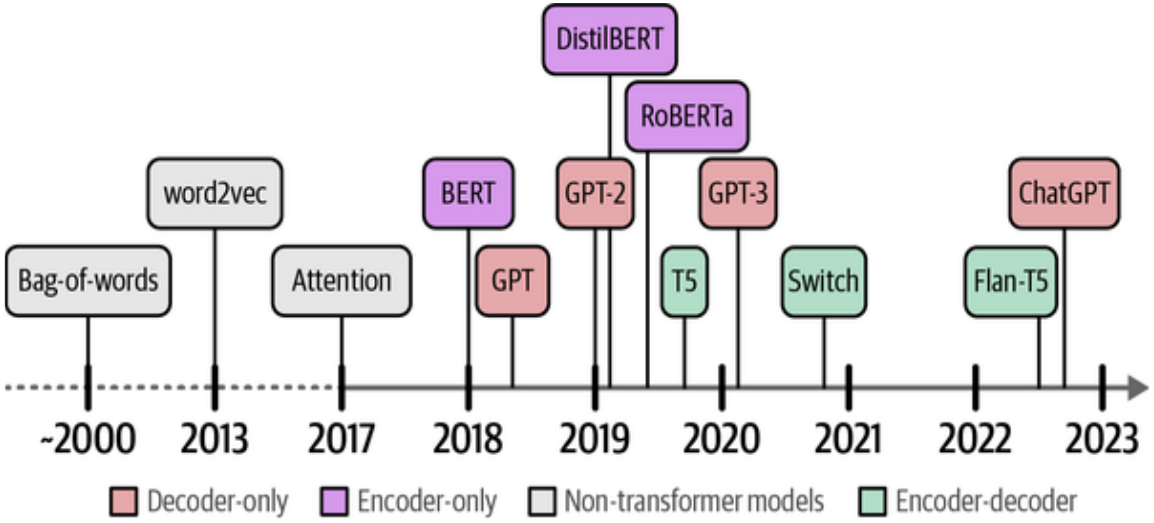
University of Petra

Institute of Analytics
Recognized Programme

307307
Introduction to Text Mining and Natural Language Processing

What is Natural Language Processing?

- Natural Language Processing (NLP) is a field at the intersection of computer science, artificial intelligence, and linguistics focused on enabling computers to understand, interpret, and generate human language.
- It forms the foundation for all language technologies we use today, from search engines to voice assistants and modern large language models.



A peek into the history of Language AI – Hands on LLMs, Jay Al Ammar Book

Why is NLP Challenging?

Human language is ambiguous: Words and sentences can have multiple interpretations

- **Lexical ambiguity**: When a word has multiple meanings
  - Example: "The bank is closed" (financial institution or riverbank)
  - Example: "The suit was expensive" (Formal set of clothes or Legal proceeding)
  - Computational challenge: Systems must select the correct word sense from multiple possibilities
- **Syntactic ambiguity**: When a sentence can be parsed in multiple ways
  - Example: "I saw the man with the telescope" (Who has the telescope?)
  - Example: "Flying planes can be dangerous" (The act of flying planes or planes that are flying?)
  - Computational challenge: Models must determine the correct grammatical structure
- **Semantic ambiguity**: When the meaning of a sentence has multiple interpretations
  - Example: "The chicken is ready to eat" (Ready to be eaten or ready to consume food?)
  - Example: "John and Mary got married last year" (To each other or to other people?)
  - Computational challenge: Systems must infer the intended meaning based on context
- **Pragmatic ambiguity**: When the intended meaning depends on context, intent, or implied information
  - Example: "It's cold in here" (Statement of fact or request to close a window/turn up heat?)
  - Example: "Do you know what time it is?" (Yes/no question or request for the time?)
  - Computational challenge: Models must understand communicative intent beyond literal meaning

Common NLP Applications

1. Sentiment analysis: Determining the emotional tone of text
    - Identifying if customer reviews are positive, negative, or neutral
    - Tracking public opinion on products, services, or topics
2. Machine translation: Translating text between languages
    - Converting text from one language to another while preserving meaning
    - Handling cultural and linguistic differences across languages
3. Text generation: Creating human-like text
    - Automatic summarization of longer documents
    - Creative writing assistance and content creation
    - Dialogue systems for conversational AI
4. Question answering: Providing answers to natural language questions
    - Extracting answers from documents or knowledge bases
    - Understanding user intent and providing relevant information
5. Information extraction: Identifying and extracting structured information from text
    - Pulling key data points from unstructured documents
    - Converting text documents into structured databases
6. Text classification: Categorizing text into predefined categories
    - Sorting emails into spam/not spam
    - Organizing documents by topic or content type

Fundamental NLP Techniques

NLTK (Natural Language Toolkit)

NLTK (Natural Language Toolkit) is a leading platform for building Python programs that work with human language data. Developed by Steven Bird and Edward Loper at the University of Pennsylvania, NLTK has become one of the most widely used libraries for natural language processing (NLP) in Python since its initial release in 2001.

Key Features of NLTK

- Comprehensive Toolkit: NLTK provides easy-to-use interfaces to over 50 corpora and lexical resources, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning.
- Educational Purpose: NLTK was designed with education in mind. It comes with comprehensive documentation and a book ("Natural Language Processing with Python") that makes it accessible for beginners.
- Built-in Datasets: NLTK includes various pre-loaded corpora like the Brown Corpus, WordNet, and many others that are useful for training and testing NLP algorithms.
- Modular Architecture: The library is organized into modules focused on specific NLP tasks, making it easy to use just the components you need.

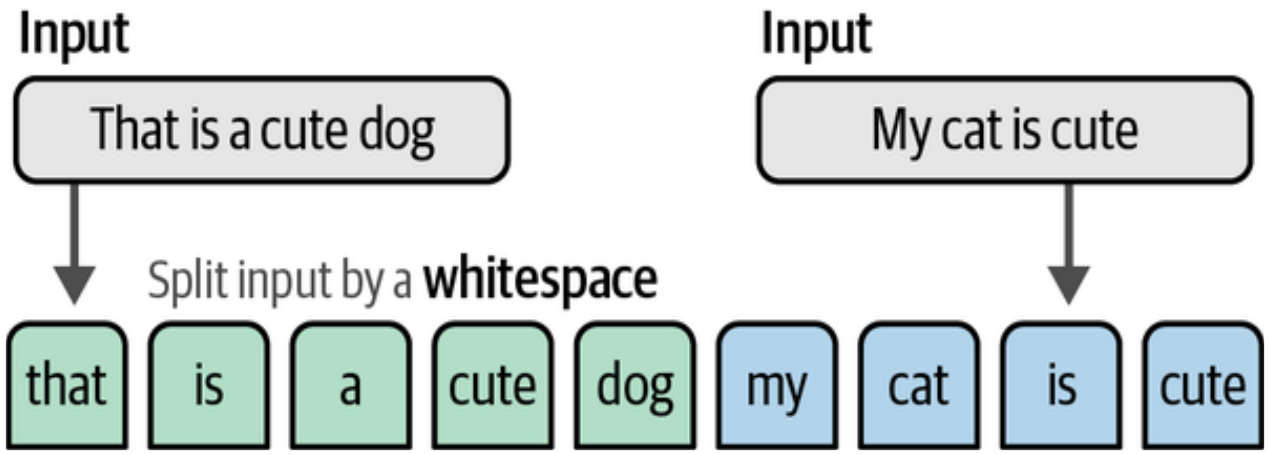
Core Functionality

- NLTK supports many essential NLP tasks:
- Tokenization: Breaking text into words, sentences, or other meaningful elements
- Part-of-speech tagging: Identifying the grammatical parts of speech for words
- Named entity recognition: Identifying names of people, organizations, locations, etc.
- Stemming and lemmatization: Reducing words to their base or root form
- Parsing: Analyzing syntactic structures of sentences
- Semantic analysis: Working with meaning representations
- Sentiment analysis: Determining positive, negative, or neutral sentiment in text

Tokenization

Tokenization: Breaking text into words, phrases, or other meaningful elements

- Word tokenization: Splitting "I love NLP." into ["I", "love", "NLP", "."]
- Subword tokenization: Breaking words into meaningful pieces (e.g., "playing" → ["play", "##ing"])
- Character tokenization: Splitting text into individual characters



Tokenize text using NLTK package

```
from nltk.tokenize import word_tokenize
#punkt is a pretrained tokenization model used for splitting text into
sentences and words.
nltk.download('punkt')

sentence = "Large language models are revolutionizing business applications."
tokens = word_tokenize(sentence)
print(tokens)
```

```
Output:
['Large', 'language', 'models', 'are', 'revolutionizing', 'business',
'applications', '.']
[nltk_data] Downloading package punkt to /home/me/nltk_data... [nltk_data]
Package punkt is already up-to-date!
```

Open in Colab

Click to Open Code

Normalization

- Converting text to a standard form to reduce variability:

```
# Lowercasing
normalized_tokens = [token.lower() for token in tokens]
print(normalized_tokens)

# Removing punctuation
import re
normalized_tokens = [re.sub(r'[^\w\s]', '', token.lower()) for token in tokens]
print(normalized_tokens)
```

The regular expression r'[^\w\s]' does the following:

- r'...': This denotes a raw string in Python. Raw strings treat backslashes \ as literal characters, which is crucial for regular expressions.
- [...]: This represents a character set.^: Inside a character set, ^ negates the set. It means "match any character not in this set.
- "\w: This matches any word character, which includes alphanumeric characters (letters and digits) and the underscore (_).
- \s: This matches any whitespace character, such as spaces, tabs, and newlines.
- Therefore, r'[^\w\s]' matches any character that is not a word character or a whitespace character. In simpler terms, it matches any non-alphanumeric, non-underscore, and non-whitespace character. This effectively targets punctuation marks, symbols, and other special characters.

Output:
['large', 'language', 'models', 'are', 'revolutionizing', 'business', 'applications', '.']['large', 'language', 'models', 'are', 'revolutionizing', 'business', 'applications', '']

Stopwords Removal

- Eliminating common words that add little meaning:

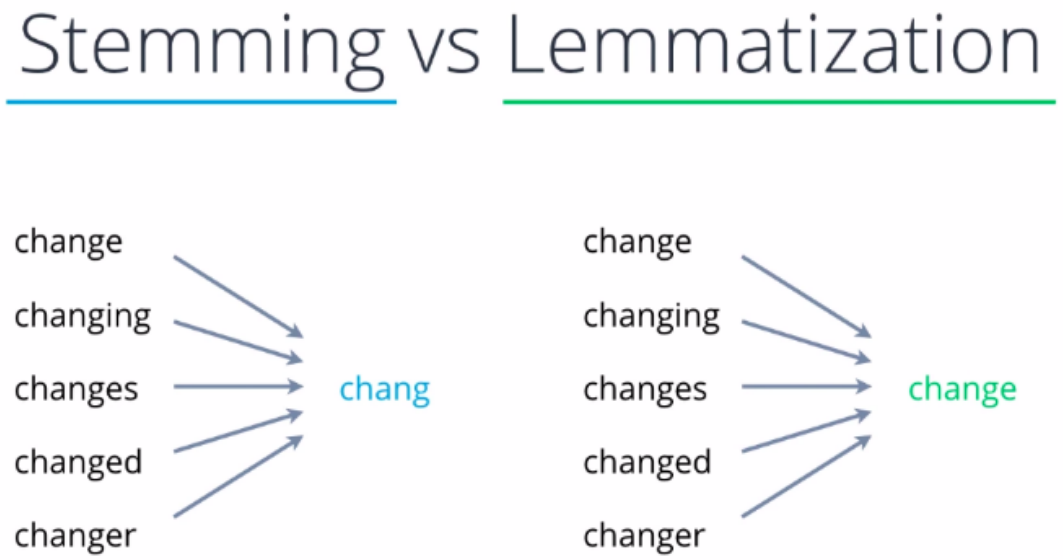
```
#nltk.download('stopwords') #stopwords are common words that are often
removed from text as they are not useful for analysis.
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
print("These words are removed from the text: ", stop_words)
filtered_tokens = [token for token in normalized_tokens if token and
token not in stop_words]
print(filtered_tokens)
```

Output:

These words are removed from the text: {'during', 'off', 'each', 'own', 'nor', "we've", "you'll", 'ourselves', 'between', 'same', 'such', 'against', 'but', "mustn't", "aren't", "wouldn't",

Stemming and Lemmatization

- Stemming and lemmatization are text normalization techniques used in Natural Language Processing (NLP) to reduce words to their base or root forms.
- They help in improving search engines, text mining, and language modeling by standardizing words with similar meanings.



Stemming

- Definition: Stemming is a rule-based process that removes prefixes and suffixes (affixes) from a word to obtain its base form, often producing words that may not be actual words.
- Method: Uses algorithms like Porter Stemmer and Snowball Stemmer.
- Example:
  - "Running" → "Run"
  - "Studies" → "Studi"
  - "Better" → "Better" (Incorrectly unchanged)
  - "Caring" → "Car"
- Limitations:
  - Often results in words that are not real words ("Studies" → "Studi").
  - Does not consider the actual meaning of words.

Lemmatization

- Definition: Lemmatization is a dictionary-based approach that reduces words to their meaningful root form (lemma), considering the word's context and part of speech.
- Method: Uses linguistic analysis and dictionaries (like WordNet Lemmatizer).
- Example:
  - "Running" → "Run"
  - "Studies" → "Study"
  - "Better" → "Good" (Correct Lemma)
  - "Caring" → "Care"
- Advantages:
  - More accurate than stemming.
  - Produces real words with proper meanings.
- Use Cases:
  - Stemming is useful when speed is more important than accuracy (e.g., search engines).
  - Lemmatization is better for semantic analysis where meaning matters (e.g., AI chatbots, NLP applications).

Stemming Example

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stemmed_tokens = [stemmer.stem(token) for token in filtered_tokens]

print(stemmed_tokens)
```

Lemmatizing Example

```
#nltk.download('wordnet')

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
lemmatized_tokens = [lemmatizer.lemmatize(token) for token in filtered_tokens]

print(lemmatized_tokens)
```

Representing Text – Converting Text into Numbers

1. Bag of Words Method.
2. TF-IDF (Term Frequency-Inverse Document Frequency)

Representing Text - Bag of Words (BoW)

Bag of Words is a fundamental technique in Natural Language Processing (NLP) that represents text as a collection of word frequencies, disregarding grammar and word order.

Core Concept

- The Bag of Words model converts text documents into numerical feature vectors by:
  - Creating a vocabulary of all unique words in the corpus
  - For each document, counting how many times each vocabulary word appears
- Simple Example:
  - Consider these two short sentences:
    - "I love natural language processing“
    - "Processing natural language is fun“
  - The vocabulary would be:
    - {"I", "love", "natural", "language", "processing", "is", "fun"}
    - Document 1 vector: [1, 1, 1, 1, 1, 0, 0]
    - Document 2 vector: [0, 0, 1, 1, 1, 1, 1]

Bag of Words (BoW)

Applications

BOW is commonly used in:

- Text classification (spam detection, sentiment analysis)
- Document clustering
- Information retrieval
- Topic modeling

Advantages

- Simple to understand and implement
- Computationally efficient
- Effective for many basic NLP tasks

Limitations

- Loses word order information
- Ignores semantics and context
- Creates high-dimensional, sparse vectors
- Struggles with out-of-vocabulary words

BoW Example

```python
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "Large language models revolutionize business.",
    "Business applications benefit from AI.",
    "Language models learn from text data."
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

['ai' 'applications' 'benefit' 'business' 'data' 'from' 'language' 'large' 'learn' 'models' 'revolutionize' 'text’]

[[0 0 0 1 0 0 1 1 0 1 1 0]

[1 1 1 1 0 1 0 0 0 0 0 0]

[0 0 0 0 1 1 1 0 1 1 0 1]]

Representing Text - TF-IDF (Term Frequency-Inverse Document Frequency)

Definition

- TF-IDF is a statistical measure that evaluates the importance of a word in a document relative to a collection of documents (corpus).
- It improves upon the basic Bag of Words model by weighing terms based on how unique they are to a specific document.

Components

1. **Term Frequency (TF):** Measures how frequently a term appears in a document
   - TF(t,d) = (Number of times term t appears in document d) / (Total number of terms in document d)
2. **Inverse Document Frequency (IDF):** Measures how important or unique a term is across all documents
   - IDF(t) = log(Total number of documents / Number of documents containing term t)
3. **TF-IDF score**: TF(t,d) × IDF(t)

TF-IDF (Term Frequency-Inverse Document Frequency)

Example

For documents:

- Doc1: "I love machine learning"
- Doc2: "I love natural language processing"
- Doc3: "Machine learning models require data"

The word "love" appears in multiple documents, so its IDF value is lower. The word "processing" appears in only one document, giving it a higher IDF value.

Applications

- **Document retrieval and search engines**: Ranking documents by relevance
- **Document classification**: Identifying distinguishing terms
- **Keyword extraction**: Finding important terms in documents
- **Text summarization**: Identifying key sentences

TF-IDF (Term Frequency-Inverse Document Frequency)

Advantages

- Reduces the impact of frequently occurring terms (like "the", "is", "of")
- Highlights terms that are distinctive to specific documents
- Relatively simple to implement and computationally efficient
- Works well for sparse data

Limitations

- Still ignores word order and semantics
- Cannot capture relationships between related terms
- May not work well for very short texts
- Struggles with synonyms and polysemy
- TF-IDF serves as a cornerstone technique in information retrieval and text mining, and remains widely used despite the emergence of more advanced embedding-based approaches.

TF-IDF (Term Frequency-Inverse Document Frequency)

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()

# Transform the corpus into a document-term matrix, each row represents a document in the corpus, each column represents a unique word in the corpus
X_tfidf = tfidf_vectorizer.fit_transform(corpus)

# Get the unique words in the corpus
print("\n Unique words in the corpus:\n")
print(tfidf_vectorizer.get_feature_names_out())

print("\n All documents in the corpus are represented as a matrix of TF-IDF features:\n")
print(X_tfidf.toarray())
```

Output:

```
Unique words in the corpus: ['ai' 'applications' 'benefit' 'business' 'data' 'from' 'language' 'large' 'learn' 'models' 'revolutionize' 'text’]

All documents in the corpus are represented as a matrix of TF-IDF features:

[[0. 0. 0. 0.3935112 0. 0. 0.3935112 0.51741994 0. 0.3935112 0.51741994 0. ]

[0.49047908 0.49047908 0.49047908 0.37302199 0. 0.37302199 0. 0. 0. 0. 0. 0. ]

[0. 0. 0. 0. 0.45954803 0.34949812 0.34949812 0. 0.45954803 0.34949812 0. 0.45954803]]
```

Practical Example

Movie Review Sentiment Analysis Pipeline

Sentiment Analysis of Movie Reviews

- This application will go through the entire text preprocessing pipeline and show how it contributes to a real-world NLP task.
- Setup:
  - Dataset: Use a small dataset of movie reviews (positive and negative) - you could use a subset of IMDB reviews or create 10-15 simple examples.
  - Visual Flow: Create a slide that shows the entire pipeline:
    - Raw Text → Tokenization → Normalization → Stop Words Removal → Stemming/Lemmatization → BoW → Classification

Open in Colab

Click to open code file

```python
import nltk
import pandas as pd
import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

# Sample movie reviews dataset
reviews = [
    {"text": "This movie was absolutely fantastic! Great acting and storyline.", "sentiment": 1},
    {"text": "I loved this film. The characters were so well developed.", "sentiment": 1},
    {"text": "Amazing cinematography and directing. One of the best films I've seen.", "sentiment": 1},
    {"text": "The acting was good but the story was too predictable.", "sentiment": 0},
    {"text": "Terrible movie. I wasted two hours of my life.", "sentiment": 0},
    {"text": "The special effects were amazing but everything else was boring.", "sentiment": 0},
    {"text": "I enjoyed the action sequences but the dialogue was poorly written.", "sentiment": 0},
    {"text": "Brilliant performance by the lead actor! Highly recommended.", "sentiment": 1},
    {"text": "So disappointing. The trailer was better than the actual movie.", "sentiment": 0},
    {"text": "A masterpiece of modern cinema. I was captivated throughout.", "sentiment": 1}
]

# Create DataFrame
df = pd.DataFrame(reviews)
print("Original Data:")
print(df.head())
print("\n")
```

```python
# Step 1: Tokenization
def tokenize_text(text):
    tokens = word_tokenize(text)
    print(f"After tokenization: {tokens}")
    return tokens

# Step 2: Normalization
def normalize_tokens(tokens):
    # Convert to lowercase and remove punctuation
    normalized = [re.sub(r'[^\w\s]', '', token.lower()) for token in tokens]
    normalized = [token for token in normalized if token]  # Remove empty strings
    print(f"After normalization: {normalized}")
    return normalized

# Step 3: Remove stop words
def remove_stopwords(tokens):
    stop_words = set(stopwords.words('english'))
    filtered = [token for token in tokens if token not in stop_words]
    print(f"After stopword removal: {filtered}")
    return filtered

# Step 4a: Stemming
def stem_tokens(tokens):
    stemmer = PorterStemmer()
    stemmed = [stemmer.stem(token) for token in tokens]
    print(f"After stemming: {stemmed}")
    return stemmed

# Step 4b: Lemmatization
def lemmatize_tokens(tokens):
    lemmatizer = WordNetLemmatizer()
    lemmatized = [lemmatizer.lemmatize(token) for token in tokens]
    print(f"After lemmatization: {lemmatized}")
    return lemmatized
```

```python
# Complete preprocessing pipeline
def preprocess_text(text, use_stemming=True):
    tokens = tokenize_text(text)
    normalized_tokens = normalize_tokens(tokens)
    no_stopwords = remove_stopwords(normalized_tokens)

    if use_stemming:
        return stem_tokens(no_stopwords)
    else:
        return lemmatize_tokens(no_stopwords)

# Demonstrate the preprocessing pipeline on one example
print("PREPROCESSING PIPELINE DEMONSTRATION:")
sample_text = df['text'][0]
print(f"Original text: '{sample_text}'")
processed_tokens = preprocess_text(sample_text, use_stemming=True)
print("\n")

# Process all reviews
df['processed_text'] = df['text'].apply(lambda x: ' '.join(preprocess_text(x, use_stemming=True)))
print("Processed Reviews:")
print(df[['text', 'processed_text']].head())
print("\n")

# Step 5: Bag of Words representation
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['processed_text'])

# Show vocabulary
print("Bag of Words Vocabulary:")
print(vectorizer.get_feature_names_out())
print("\n")

# Show BOW representation for first few documents
print("BOW Matrix (First 3 Documents):")
bow_df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
print(bow_df.head(3))
print("\n")
```

```python
# Step 6: Simple classifier
X_train, X_test, y_train, y_test = train_test_split(
    X, df['sentiment'], test_size=0.3, random_state=42
)

# Train a Naive Bayes classifier
clf = MultinomialNB()
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Evaluate
print("SENTIMENT CLASSIFICATION RESULTS:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Demonstrate classification on new examples
def predict_sentiment(text):
    # Preprocess
    processed = ' '.join(preprocess_text(text, use_stemming=True))
    # Transform to BOW
    bow = vectorizer.transform([processed])
    # Predict
    prediction = clf.predict(bow)[0]
    sentiment = "Positive" if prediction == 1 else "Negative"
    return sentiment

print("\nTRY WITH NEW EXAMPLES:")
new_reviews = [
    "This movie exceeded all my expectations. The plot was incredible!",
    "What a waste of time. The worst movie I've seen this year."
]

for review in new_reviews:
    sentiment = predict_sentiment(review)
    print(f"Review: '{review}'")
    print(f"Predicted sentiment: {sentiment}\n")
```

Language Models

Language Models: N-grams

Language models are statistical or neural models that capture the probability distribution of words or tokens in a language.

They predict the likelihood of a sequence of words occurring together, enabling machines to understand and generate human language.

Types of Language Models

1) Statistical Language Models

- **N-gram Models**: Predict the next word based on the previous N-1 words
  - Unigram: P(word) - individual word probabilities
  - Bigram: $P(word_2|word_1)$ - probability of $word_2$ given $word_1$
  - Trigram: $P(word_3|word_1,word_2)$ - probability of $word_3$ given $word_1$ and $word_2$

2) Neural Language Models

- **RNN/LSTM-based**: Process sequences by maintaining hidden states
  - Better at capturing long-range dependencies than n-grams
  - Examples: Early versions of Google Translate
- **Transformer-based**: Use attention mechanisms to process entire sequences
  - BERT: Bidirectional, focuses on understanding context
  - GPT: Autoregressive, focuses on text generation
  - T5, RoBERTa, ELECTRA: Refinements of transformer architecture

Core Concepts

1. **Tokenization**: Breaking text into tokens (words, subwords, or characters)
2. **Embeddings**: Dense vector representations of words/tokens
   - Word2Vec, GloVe: Context-independent embeddings
   - Contextual embeddings: Different representations based on context
3. **Perplexity**: Evaluation metric measuring how well a model predicts a sample
   - Lower perplexity = better prediction
4. **Fine-tuning**: Adapting pre-trained models to specific tasks

Applications

- **Text Generation**: Writing assistance, content creation
- **Machine Translation**: Converting text between languages
- **Summarization**: Condensing documents while preserving meaning
- **Question Answering**: Understanding and responding to queries
- **Text Classification**: Sentiment analysis, topic categorization
- **Dialogue Systems**: Conversational agents and chatbots

Implementing N-grams

```python
import random
from collections import defaultdict

def build_ngram_model(text, n=2):
    """Build an n-gram language model from text."""
    tokens = word_tokenize(text.lower())
    ngrams_dict = defaultdict(list)

    # Create dictionary of n-grams and possible next words
    for i in range(len(tokens) - n):
        current_ngram = tuple(tokens[i:i+n])
        next_word = tokens[i+n]
        ngrams_dict[current_ngram].append(next_word)

    return ngrams_dict

def generate_text(model, seed, length=20):
    """Generate text using the n-gram model."""
    current = seed
    result = list(seed)

    for _ in range(length):
        if current in model:
            # Randomly select a possible next word
            next_word = random.choice(model[current])
            result.append(next_word)
            # Update current n-gram
            current = current[1:] + (next_word,)
        else:
            # If current n-gram is not in model, break
            break

    return ' '.join(result)

# Sample text corpus
corpus = """Large language models are transforming how businesses operate.
These models can understand language, generate text, and perform various tasks.
Businesses use language models for customer service, content creation, and data analysis.
Language models learn patterns from vast amounts of text data."""

# Build a bigram model
bigram_model = build_ngram_model(corpus, 2)

# Generate text using the model
seed = ('language', 'models')
generated_text = generate_text(bigram_model, seed)
print(generated_text)
•# Output might be: "language models learn patterns from vast amounts of text data businesses use language models for customer service content creation and data analysis"
```

Limitations of N-gram Models

- Limited context: Only consider a fixed number of previous words
- Data sparsity: Many valid word combinations don't appear in training data
- No semantic understanding: Model captures statistical patterns but not meaning
- Memory intensive: Storing all possible n-grams requires significant space
- No long-range dependencies: Cannot capture relationships between distant words

Perplexity

Perplexity measures how confused a language model is when predicting text.

What It Means

- **Lower perplexity** = Model is more confident and accurate
- **Higher perplexity** = Model is more uncertain and confused

Simple Example

Imagine you're trying to predict the next word in: "I went to the _____"

- A good model might assign high probability to words like "store," "park," "beach"
- A confused model might spread probabilities evenly across thousands of words

Perplexity essentially counts how many reasonable options the model is considering. If the model has a perplexity of 10, it's like the model is equally confused between 10 possible words.

Real-World Comparison

If your phone keyboard suggests 3 very likely next words, it has low perplexity. If it suggests random unrelated words, it has high perplexity.

Why It Matters

Think of perplexity as a "confusion score" that helps researchers:

- Compare different language models
- Check if a model is improving during training
- Identify when a model works better for certain types of text
- The best language models today have much lower perplexity than older models, which is why they produce more natural-sounding text.